

Risk-Seeking EVaR in Distributional Deep RL

Porting multi-timescale SPSA EVaR optimisation to a distributional actor–critic: method, implementation, measurements, and the state of the evidence

Status report for collaborators

This report documents a port of the TMLR method *Risk-Seeking Reinforcement Learning via Multi-Timescale EVaR Optimization* (SPSA-based) into a distributional deep RL setting, and states precisely what has been established, what has been refuted, and what remains open. It is deliberately explicit about negative and null results: several plausible hypotheses were tested and failed, and two implementation faults produced numbers that looked like findings before they were caught. Both are documented here, because the diagnostics that caught them are now part of the method.

1 Executive summary

The work has three claims to defend, inherited from the source paper. Their status:

ID	Claim	Status	Evidence
C1	The method optimises the intended objective: EVaR_α of trajectory returns rises, α moves the tail monotonically, $\alpha = 1$ recovers risk-neutral	CLOSED (exactly)	Exact regret 0.00% at all seven α on the lottery gridworld with a perfect critic; $\alpha = 1$ control exact
C2	It is cheaper than SPSA at matched sample budget	ARGUED	Settled analytically by decision, not by a head-to-head: SPSA spends $2N_t$ rollouts per update to estimate a scalar, this spends one backprop. The paper’s published numbers stand for the SPSA side
C3	The per-state approximation is sound	CLOSED (reformulated)	Was not an approximation gap but a <i>formulation error</i> . State-value form costs up to 86.35%; action-value form is exact (0.00%)

The one-line state of play. The operator is correct and now has a learner capable of standard continuous-control benchmarks. What is *not* yet established is that EVaR outperforms the standard distortion risk measures; on every environment tested so far the arms tie, and in each case we can now say *why* they tie, which is a precondition for fixing it rather than a result.

What changed most recently. A latent fault made the EVaR estimate scale-dependent: the dual variable $x^* = 1/\beta^*$ carries units of return, so a fixed search interval silently degenerated EVaR into fixed- β entropic utility on any environment whose returns are small. Measured error reached 1100%. This is fixed, instrumented, and — because the fix is essentially “adapt β to the distribution” — is the most promising seed for the intended contribution.

2 Background: what is being ported

2.1 The source method

The source paper optimises

$$J_{\text{EVaR}}(\theta) = \text{EVaR}_\alpha[R(\tau)], \quad \tau \sim \pi_\theta, \quad (1)$$

with a two-timescale stochastic-approximation recursion. Because no gradient of J_{EVaR} is available, the policy is perturbed with SPSA, requiring $2N_t$ rollouts per policy update purely to estimate a scalar.

2.2 Entropic Value-at-Risk

For a return Z and confidence $\alpha \in (0, 1]$,

$$\text{EVaR}_\alpha[Z] = \min_{\beta > 0} \frac{1}{\beta} \left(\log \mathbb{E}[e^{\beta Z}] - \log \alpha \right). \quad (2)$$

Under the reparameterisation $x = 1/\beta$ this becomes minimisation of

$$G(x) = x \left(\log \mathbb{E}[e^{Z/x}] - \log \alpha \right), \quad (3)$$

which is strongly convex on a compact interval, so G' is monotone and a sign change brackets the minimiser. Two properties drive nearly every design decision that follows:

1. **Translation equivariance.** $\text{EVaR}_\alpha[c + Z] = c + \text{EVaR}_\alpha[Z]$ for scalar c . A constant passes through the tilt untouched.
2. **x^* has units of return.** G is homogeneous of degree one: rescaling Z by λ rescales both EVaR and x^* by λ . Empirically $x^* \approx 0.34 \text{sd}(Z)$ for Gaussian returns (Fig. 1).

2.3 What the port replaces

A distributional critic supplies an explicit, differentiable return distribution, so (3) can be minimised directly per state and differentiated, replacing the outer SPSA loop. One backpropagation replaces $2N_t$ rollouts. The remainder of this report is about what that substitution requires to be correct.

3 Method

3.1 The formulation error: why the critic must be an action-value

This is the single most consequential finding and it was originally misdiagnosed as an approximation gap.

The natural per-state advantage is $A(s, a) = r + \text{EVaR}_\alpha(Z(s'))$. This is *risk-neutral in the reward*. By translation equivariance, a *sampled scalar* reward r inside the tilt is identical to r outside it, so $\text{EVaR}_\alpha(r + Z(s')) \equiv r + \text{EVaR}_\alpha(Z(s'))$. At a terminal step $Z(s')$ is degenerate, α cannot enter at all, and the comparison reduces to safe-reward against lottery *mean*. That makes the previous step deterministic, and the collapse propagates backwards: the fixed point is the risk-neutral policy at every α .

The tilt reaches the reward only when the critic represents the reward’s *distribution*, i.e. $Z(s, a)$. Measured exactly on the lottery gridworld with a perfect critic and greedy improvement (no learning, so the numbers are exact):

Advantage form	α values correct	Worst exact regret
$r + \text{EVaR}_\alpha(Z(s'))$ (state-value)	1 of 7	86.35%
$\text{EVaR}_\alpha(Z(s, a))$ (action-value)	7 of 7	0.00%

The mechanism is visible in the solved duals: the trajectory objective decouples at one *global* x^* , while the per-state solve uses a different $x^*(s)$ at each state. At $\alpha = 0.10$, global $x^* = 36.54$ while the per-state solve gives 36.54, 36.54, 0.00 at the three decision columns — the terminal column collapses to zero, which is precisely the degeneracy above.

3.2 The distributional critic

$Z(s, a)$ is represented by implicit quantiles (IQN) rather than a fixed categorical support. The reason is concrete rather than aesthetic: C51 requires $v_{\min}, v_{\max}$ in the units the critic actually represents — *discounted* return, not undiscounted episode return. Getting this wrong left ≈ 9 of 51 atoms carrying any mass on CartPole and ≈ 5 of 51 on InvertedPendulum, and since EVaR is a tail statistic read off exactly that histogram, the operator was being fed a 9-point support. Quantiles have no support to mis-size and place resolution where the distribution is.

The critic embeds state-action and quantile level separately and combines them multiplicatively:

$$Z_\tau(s, a) = f\left(\psi(s, a) \odot \phi(\tau)\right), \quad \phi_j(\tau) = \text{ReLU}\left(\sum_i \cos(\pi i \tau) w_{ij} + b_j\right), \quad (4)$$

trained with the quantile Huber loss over all $(\tau_i, \text{target}_j)$ pairs.

3.3 The inner solve

Bisection, not Newton. The original projected-Newton solve never converged. When the tilted measure collapses onto one atom, $\text{Var}_{Q_x} \rightarrow 0$, the curvature $G'' = \text{Var}_{Q_x}/x^3$ underflows its clamp, the step G'/G'' explodes, is projected onto a bound, and cycles between the bounds forever. The returned x^* was whichever bound the step-count parity landed on — constant at 0.301 across genuinely different return distributions — and EVaR was inflated by 25–590%, with the error *growing in the scale of returns*, i.e. in the same direction as C1 itself. Bisectioning the monotone G' in $\log x$ converges unconditionally and now matches brute-force minimisation to 0.00%.

Gradients by the envelope theorem. x^* is detached. Since $\partial G/\partial x = 0$ at the optimum, $dG/d\theta = \partial G/\partial\theta$ evaluated at x^* , so gradients into the critic are exact without unrolling the solver — and, unlike an unrolled loop, cannot be corrupted by the iteration’s own conditioning. Checked against finite differences at 5×10^{-9} .

Zero-probability atoms are masked, not floored. A `clamp_min(1e-12)` does not guard $\log 0$ harmlessly: it *invents* mass 10^{-12} on atoms whose true probability is zero, and the tilt $e^{Z/x}$ multiplies that fake mass by $e^{z_{\max}/x}$. On a C51 support up to 500 with $x \approx 11$ that is a factor of e^{45} , enough for a floored atom to dominate Q_x and pull EVaR up by 13.3%. Genuine zeros are masked to $-\infty$ instead.

The saturated regime is returned exactly. $G'(0^+) = \log(p_{\max}/\alpha)$ where $p_{\max}$ is the mass on the largest atom, so once $p_{\max} \geq \alpha$ the objective is non-decreasing everywhere and the infimum sits at $x \rightarrow 0$, where $G \rightarrow z_{\max}$. EVaR then *is* the maximum. Bisectioning anyway lands on $x_{\min}$ and returns $z_{\max} + x_{\min} \log(1/\alpha)$, breaking the invariant $\mathbb{E}[Z] \leq \text{EVaR}_\alpha[Z] \leq \max(Z)$.

3.4 Scale-adaptive dual interval (new)

Because x^* has units of return, a fixed interval $[x_{\min}, x_{\max}]$ is only valid at one return scale. Figure 1 shows the consequence.

At the bound, EVaR equals mean $+ x_{\min} \log(1/\alpha)$ exactly — that is fixed- β entropic utility, which is precisely the baseline the dual solve exists to beat. The interval is now derived per row as $[10^{-3} \text{sd}(Z), 10^3 \text{sd}(Z)]$, with the absolute bounds retained as fallbacks for degenerate rows. Validated against brute-force minimisation over a dense logarithmic grid: maximum relative error 5×10^{-6} from $\text{sd}(Z) = 100$ down to $\text{sd}(Z) = 0.001$.

Why this matters beyond being a bugfix. It is a measured demonstration that the tilt strength must adapt to the distribution it is applied to. That is the same argument as adaptive β , with a failure mode behind it instead of an intuition.

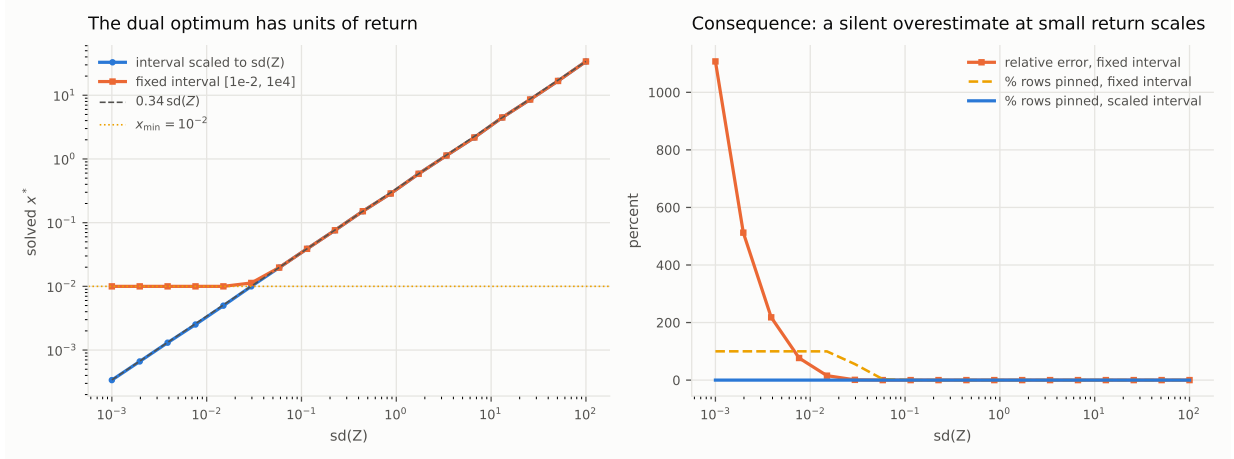


Figure 1: **Left:** the solved dual x^* tracks $0.34\text{sd}(Z)$ over five orders of magnitude. With a fixed interval it flattens onto $x_{\min} = 10^{-2}$ as soon as the true optimum falls below it. **Right:** the resulting relative error in EVaR, reaching 1100% at $\text{sd}(Z) = 10^{-3}$, alongside the fraction of rows pinned at a bound. With the interval derived per row from $\text{sd}(Z)$, nothing pins at any scale.

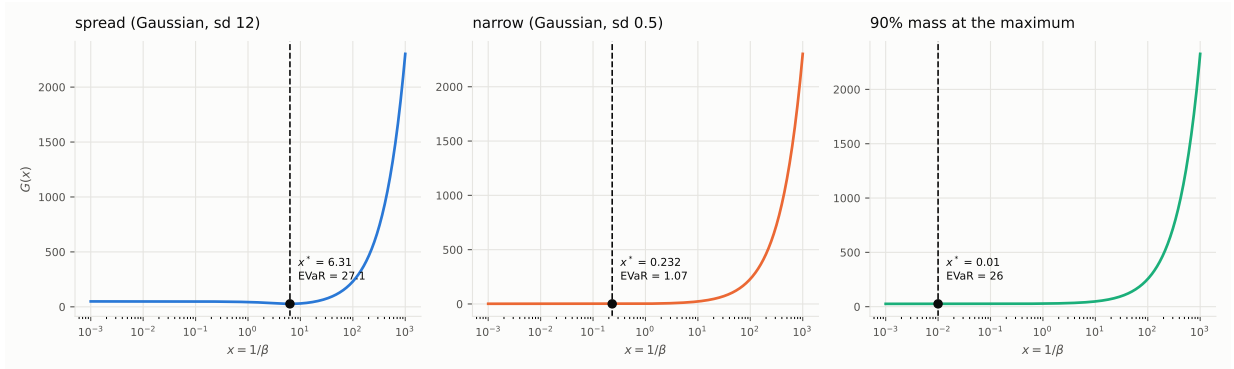


Figure 2: The dual objective $G(x)$ for three distribution shapes with the solved optimum marked. Left and centre: an interior minimum, correctly located at very different x for the same α . Right: with 90% of the mass at the maximum the optimum is driven toward $x \rightarrow 0$ and EVaR approaches $\max(Z)$ — the saturated regime, which is detected and returned exactly rather than bisected.

3.5 Solver budget

4 The learner

4.1 Rationale

Two earlier learners failed for recorded reasons. The n -step A2C could not do continuous control at all — on **SafetyPointGoal1** with cost priced at zero it sat at random-policy reward for 1500 episodes. The on-policy PPO learner works but is sample-hungry, and this literature runs continuous control off-policy. The current learner is a risk-sensitive DSAC.

Off-policy with replay. The comparisons that matter on continuous control are off-policy; it also means the tail is estimated from the whole buffer rather than one rollout, which is the regime the risk measure needs.

IQN action-value critic. $\text{EVaR}(Z(s, a))$, never $r + \text{EVaR}(Z(s'))$, for the reason in §3.1.

Twin critics, minimum over the risk value. Standard overestimation control, applied to the quantity the actor actually maximises.

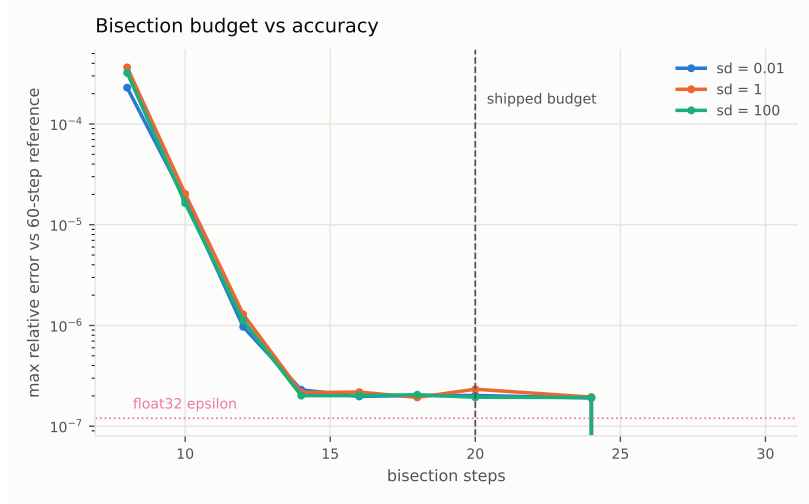


Figure 3: Accuracy against a 60-step reference. The shipped budget of 20 bisections sits at float32 machine epsilon across return scales; 30 bought nothing measurable while costing ≈ 12 kernel launches each.

Tanh-squashed actor. A clamped Gaussian returns the density of the *unclamped* sample, so any objective comparing densities of executed actions evaluates them at a boundary where the density swings violently once the mean drifts outside the bound. In the PPO learner this produced approximate KL of 64 and then 1.4×10^{16} , killing runs outright. Tanh squashing has no boundary; the log-probability carries the exact Jacobian correction $\log(1 - \tanh^2 u) = 2(\log 2 - u - \text{softplus}(-2u))$.

Automatic entropy temperature. Hand-tuning wasted real time earlier: at 10^{-3} and 10^{-2} policy entropy collapsed identically, because the coefficient is scaled against *normalised* advantages. The fix was removing the constant, not choosing a better one.

Pluggable risk functional. `--risk mean` is a true risk-neutral control sharing critic, actor, data and seeds. It is the only thing that licenses attributing a difference to the risk measure.

4.2 Update

Per environment step, one gradient update. Critic target (with d the *termination* flag only, so time-limit truncation still bootstraps):

$$y_j = r - \lambda c + \gamma(1 - d) \left(\min_{k \in \{1, 2\}} Z_{k, \tau_j}^{\text{targ}}(s', a') - \eta \log \pi(a' | s') \right), \quad a' \sim \pi(\cdot | s'), \quad (5)$$

regressed with the quantile Huber loss. Actor objective, maximising the risk value rather than the mean:

$$\mathcal{L}_\pi = \mathbb{E}_s \left[\eta \log \pi(a | s) - \min_{k \in \{1, 2\}} \rho_\alpha(Z_k(s, a)) \right], \quad a \sim \pi(\cdot | s), \quad (6)$$

with ρ_α the selected risk functional (EVaR $_\alpha$, mean, CVaR, Wang, CPW, fixed- β entropic, or mean-variance).

4.3 Constraint: $\alpha > 1/K$

On a K -sample empirical measure the top sample carries mass $1/K$. Once that reaches α the dual optimum runs to $x_{\min}$ and EVaR collapses onto the sample maximum. With $K = 64$ risk quantiles the floor is $\alpha > 0.0156$. This is checked at construction and raises rather than silently saturating. The same constraint binds independently on evaluation sample size; an early $\alpha = 0.01$ sweep was invalid in all 500 of its evaluation rows for this reason.

5 Hyperparameters and implementation

Parameter	Value	Note
<i>Risk operator</i> (<code>evan_deeprl/risk/evan.py</code>)		
α	0.1	risk level; $\alpha = 1$ is the exact risk-neutral control
<code>solver_steps</code>	20	bisections; float32 epsilon (Fig. 3)
<code>auto_scale_bounds</code>	True	per-row interval from $\text{sd}(Z)$
<code>rel_x_min, rel_x_max</code>	$10^{-3}, 10^3$	multiples of $\text{sd}(Z)$
<code>x_min, x_max</code>	$10^{-2}, 10^4$	absolute fallback for degenerate rows
gradient path	envelope theorem	x^* detached; FD-checked at 5×10^{-9}
<i>IQN action-value critic</i> (<code>distributional/iqn.q.py</code>)		
hidden sizes	(256, 256)	state-action encoder, ReLU
embedding dim	128	
$n_{\cos}$	64	cosine basis for τ
Huber κ	1.0	
K critic / target / risk	32 / 32 / 64	risk uses more samples; $\alpha > 1/K$
<i>Actor</i> (<code>policies/squashed_gaussian.py</code>)		
hidden sizes	(256, 256)	ReLU
$\log \sigma$ clamp	$[-20, 2]$	
squashing	tanh	exact Jacobian correction
<i>DSAC</i> (<code>agents/dsac_evan.py</code>)		
optimiser	Adam	
learning rates	3×10^{-4}	actor, critic, entropy temperature
batch size	256	
replay capacity	10^6	
γ	0.99	
τ (polyak)	0.005	
updates per env step	1	
target entropy	$-\dim(\mathcal{A})$	auto-tuned temperature
warm-up (<code>start_steps</code>)	2000–5000	uniform random actions
bootstrap flag	terminated only	truncation still bootstraps
cost pricing	$r_{\text{eff}} = r - \lambda c$	$\lambda = 0$ in the runs reported here

6 Diagnostics and invariants

Every one of these was added because it caught a real fault. They are logged each update.

Diagnostic	Healthy value	What it catches
<code>at_bound_frac</code>	0 for $\alpha < 1$	Dual solve landing on its interval: EVaR has degenerated into fixed- β entropic utility. Sat at 1.0 for every update of every run while Newton was broken
<code>top_mass_frac</code>	$\approx 1/K = 0.0156$	Mass within 0.05sd of the maximum. High means the critic produced a near-degenerate Z and EVaR is reporting its maximum, not a tail average
<code>z_sd_mean</code>	task-dependent	Spread available to the risk measure. If small relative to return scale, all risk attitudes tie <i>by construction</i>
<code>saturated_frac</code>	0	$p_{\max} \geq \alpha$: EVaR <i>is</i> $\max(Z)$ and is returned exactly
$\mathbb{E}[Z] \leq \text{EVaR} \leq \max Z$	always	Bounds invariant; exposed the saturated regime

A cautionary note on diagnostics. The first version of `at_bound_frac` was itself wrong: bisection drives `lo` and `hi` onto x^* , so comparing the solution against the post-loop bounds compares it against itself, and every solve reported as bound-hitting. It was caught by calibrating on distributions with known answers — a Gaussian with $\text{sd} = 12$ reported `at_lower` = 1.00 while $x^* = 4.05$ sat plainly interior. *Diagnostics require the same validation as the code they watch.*

7 Experiments and results

7.1 Exact testbed: the lottery gridworld

A three-segment gridworld where each segment offers a safe payoff or a lottery, and trajectory EVaR is exactly computable by enumeration. It exists because claims about tail statistics need a setting where the ground truth is known rather than estimated.

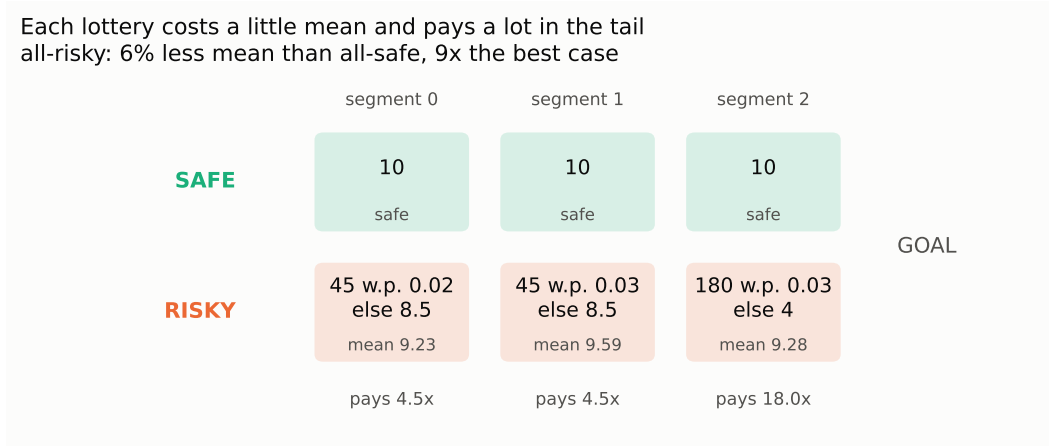


Figure 4: The lottery gridworld. Each risky segment sacrifices a little mean for a large upper tail: taking all three lotteries gives up 6% of the mean return for 9× the best case. This asymmetry is deliberate — a risk-seeking method must be able to *gain* something by accepting variance, or the benchmark cannot distinguish it from a broken one.

C3, exactly. With a perfect critic and greedy improvement, so that any gap is a property of the method and not of an optimiser:

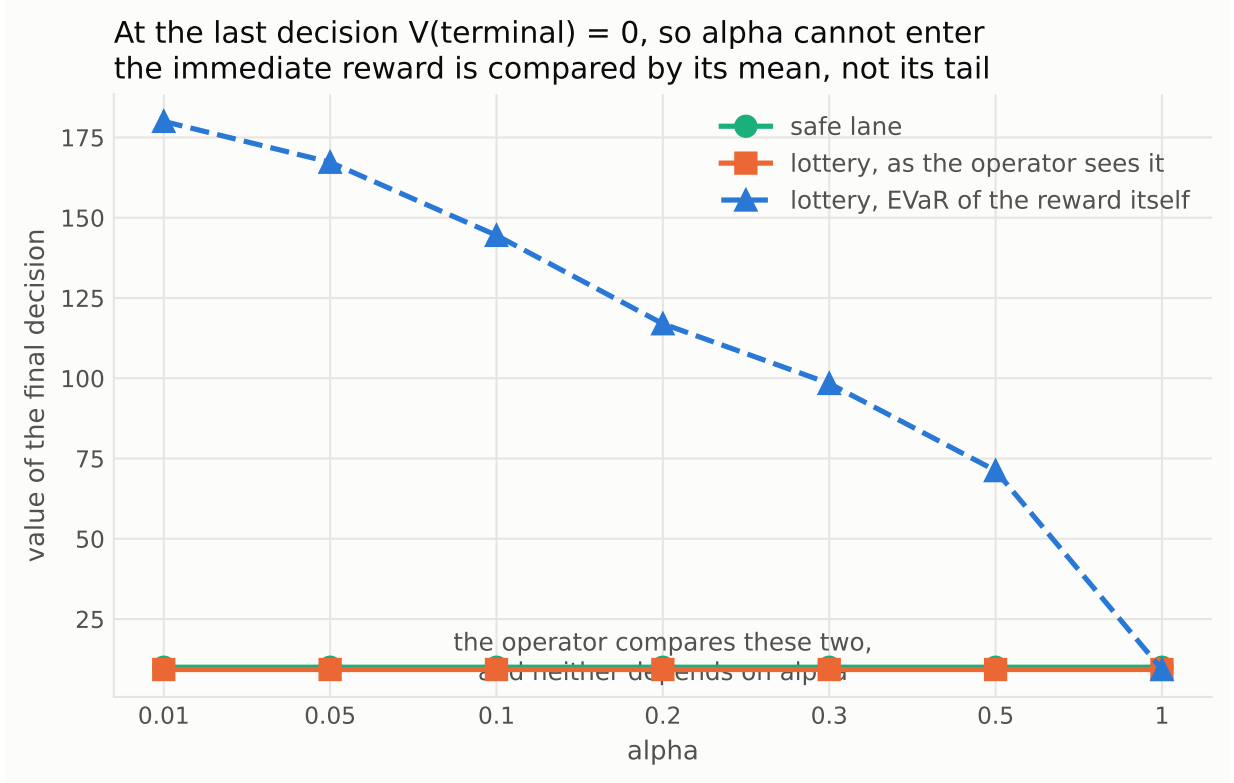


Figure 5: The C3 mechanism: how the state-value operator’s tilt fails to reach the sampled reward, and where the per-state dual departs from the global one.

α	Trajectory optimum	Per-state fixed point	EVaR opt	EVaR fp	Regret
0.01	RISKY-RISKY-RISKY	SAFE-SAFE-SAFE	219.774	30.000	86.35%
0.05	SAFE-RISKY-RISKY	SAFE-SAFE-SAFE	187.867	30.000	84.03%
0.10	SAFE-RISKY-RISKY	SAFE-SAFE-SAFE	164.835	30.000	81.80%
0.20	SAFE-RISKY-RISKY	SAFE-SAFE-SAFE	137.157	30.000	78.13%
0.30	SAFE-RISKY-RISKY	SAFE-SAFE-SAFE	118.482	30.000	74.68%
0.50	SAFE-SAFE-RISKY	SAFE-SAFE-SAFE	91.223	30.000	67.11%
1.00	SAFE-SAFE-SAFE	SAFE-SAFE-SAFE	30.000	30.000	0.00%

The state-value form returns the risk-neutral policy at *every* α — the signature of the degeneracy in §3.1. The action-value form reaches the trajectory optimum at all seven values with 0.00% regret.

Caveat, stated deliberately. 0.00% is exact *on this environment*, where segments are independent and the objective decouples. It establishes that the state-value form is the source of the bias and that the action-value form removes it here — not that per-state tilting is exact in general.

7.2 Path dependence: a hypothesis tested and refuted

The literature holds that per-state risk is a proxy for trajectory-level risk and can be suboptimal. The natural place for that to bite is when segments are *dependent*, so the decoupling above no longer applies. We broke independence by making each lottery depend on the lane taken previously:

	Independent segments		Path-dependent segments	
	state-value	action-value	state-value	action-value
Worst regret	86.35%	0.00%	81.29%	0.00%

REFUTED The action-value form stays exact at every α even with dependence. The premise for a trajectory-consistency contribution does not hold on this construction, and we should not build on it.

7.3 Can the risk measures be told apart at all?

A benchmark is worthless if the methods it compares want the same policy. An exhaustive search over 28 candidate lottery configurations, scoring six risk measures by their exact optimal policies, found configurations where **5 of 6 measures select distinct optima**. Example:

Measure	Optimal policy	Value
mean	SAFE-SAFE-SAFE	30.00
EVaR _{0.1}	RISKY-RISKY-SAFE	83.08
CVaR _{0.1}	RISKY-RISKY-RISKY	64.19
Wang _{0.75}	SAFE-RISKY-RISKY	38.14
entropic _{0.05}	SAFE-RISKY-SAFE	41.56
mean-variance ₁	SAFE-RISKY-SAFE	43.15

This matters for interpreting the null results below: the measures *are* separable in principle. Where they tie, that is a property of the environment, not a proof that EVaR is redundant.

7.4 Pendulum-v1: learner validation

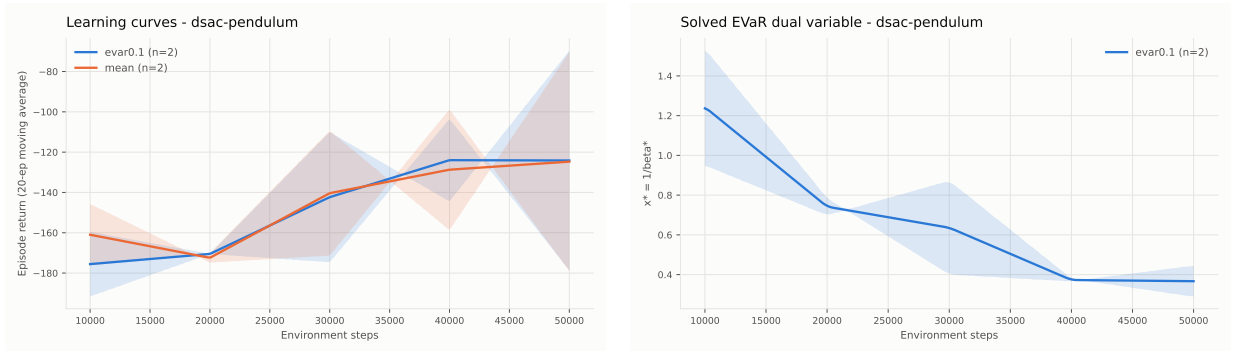


Figure 6: Pendulum: return (left) and the solved dual x^* (right), EVaR arm against the matched risk-neutral control.

Seed	mean control	EVaR _{0.1}	Paired gap
0	-96.995	-96.365	+0.63
1	-152.436	-151.919	+0.52

Random policy is ≈ -1200 , so both arms solve the task. **Read this table by seed, not by arm.** Seed variance is 55 return points while the within-seed gap is 0.6 and 0.5 — the arms track each other run for run, which is much harder to obtain by accident than a matching average.

A tie here is the correct outcome, not a null result. Pendulum’s return spread is policy noise rather than risk in the world, so there is no upper tail to trade for. This is the same reasoning that retired CartPole, where the risk-neutral and risk-seeking optima are provably the

same policy, the return is capped at 500 and integer-valued (censoring the very tail the operator is defined on), and a measured sweep produced non-monotone tail statistics with the risk-neutral control *highest*. What Pendulum validates is plumbing: that the operator is wired in, that x^* solves to an interior value (0.37–0.41) rather than pinning, and that switching the functional does not destabilise the learner.

7.5 SafetyPointGoal1-v0

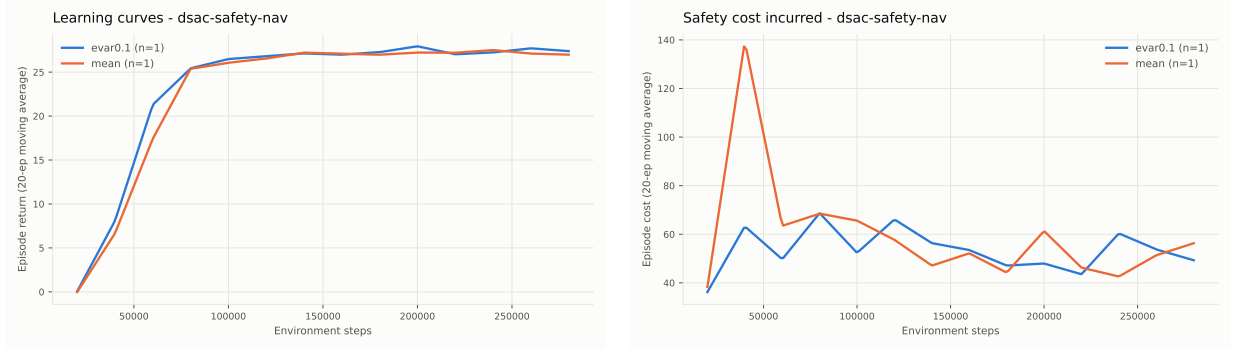


Figure 7: SafetyPointGoal1, 300k steps, cost priced at zero ($\lambda = 0$): return (left) and incurred safety cost (right).

Arm	Final return	Cost	Throughput
mean control	26.986	56.30	62.7 steps/s
EVaR _{0.1}	27.391	49.30	49.0 steps/s

Result 1: the learner solves the benchmark. The n -step A2C sat at random-policy reward here for 1500 episodes. This reaches ≈ 27 and holds. The continuous-control ladder that blocked everything downstream is open.

Result 2: this environment cannot discriminate risk attitudes, and that is now measured rather than suspected. A probe run logging the shape of $Z(s, a)$ throughout training:

Step	Return	x^*	sd(Z)	top-mass	at-bound
5,000	−0.00	0.0451	0.0754	0.017	0.00
10,000	0.30	0.0168	0.0358	0.017	0.00
20,000	0.49	0.0084	0.0218	0.019	0.00
30,000	5.73	0.0173	0.0240	0.017	0.00
35,000	10.80	0.0122	0.0228	0.017	0.00

Read in three parts. **at-bound** holds at 0, so the dual solve finds interior optima and EVaR is not degenerating. **top-mass** sits at 0.017 against a floor of $1/K = 0.0156$, so the distribution is spread rather than piled at its maximum and EVaR is a genuine tail average. *Both operator checks pass*, which is what makes the third reading trustworthy: $\text{sd}(Z) \approx 0.022$ against episode returns reaching ≈ 27 — under 1% spread. The per-state return distribution is nearly deterministic, so no risk attitude has anything to trade, and the 1.5% gap between arms is exactly what that predicts.

This is the *stochasticity trap*: on a near-deterministic task, $\text{EVaR} \approx \text{CVaR} \approx \text{mean}$ and every risk attitude ties, so any “risk-seeking wins” claim is unfalsifiable — and so is a “risk-seeking ties” one. Pricing cost at zero is part of the cause, since it removes the risk/reward tension by construction. But the deeper point stands: navigation with dense shaped reward is not a risky task.

8 Performance engineering

Measurement	Before	After
A2C throughput, single run	336 steps/s	1358 steps/s
A2C throughput, 8 parallel runs	7.5 steps/s	1314 steps/s
DSAC EVaR, Pendulum, GPU	49 steps/s	75 steps/s
Usable C51 atoms (CartPole)	≈ 9 of 51	≈ 42 of 51

Where the time actually goes — profiled rather than assumed, and the obvious reading was wrong. EVaR looked twice as expensive as the mean on GPU, but on CPU it costs only 12% (35 vs 40 updates/s), and the MuJoCo simulator runs at 617 steps/s, $30\times$ faster than the learner either way. The cost is therefore *kernel launches*, not arithmetic: on GPU the 256×256 nets are nearly free and what remains is the bisection’s ≈ 12 launches per step. Addressed by cutting `solver_steps` $30 \rightarrow 20$ and solving both twin critics in one stacked $(2B, K)$ call — verified bit-identical, since rows are independent inside the solver.

Infrastructure notes. `safety-gymnasium 1.0.0` pins `gymnasium 0.28.1` against the main environment’s 1.3.0, so it requires a separate interpreter; that `venv` needed its own CUDA torch build (`2.13.0+cu126`). `MUJOOCO_GL` must be left *unset*: an EGL or OSMesa context in the same process as a CUDA context segfaults (rc 139) on both backends. Training reads lidar vectors rather than pixels, so no GL is required.

9 Status: what is closed, open, and refuted

9.1 Closed

1. **The C3 formulation.** State-value tilting is biased (86.35%); the action-value form is exact (0.00%) on the exact testbed. **CLOSED**
2. **Solver correctness.** Bisection converges unconditionally, matches brute force to 0.00%, gradients validated at 5×10^{-9} . **CLOSED**
3. **Scale invariance of the operator.** Adaptive dual interval; verified to 5×10^{-6} across five orders of magnitude of return scale. **CLOSED**
4. **A learner for continuous control.** DSAC solves Pendulum and SafetyPointGoal1. **CLOSED**
5. **Support sizing, zero-mass atoms, saturated regime, $\alpha > 1/K$.** All identified, fixed, and instrumented. **CLOSED**
6. **Environment screening criterion.** $\text{sd}(Z)$ relative to return scale, measurable in a 40k-step probe. **CLOSED**

9.2 Refuted (recorded so they are not re-attempted)

1. Advantage normalisation as the source of risk-taking behaviour. **REFUTED**
2. Tail estimation as EVaR’s failure mode. **REFUTED**
3. Path dependence breaking per-state exactness. **REFUTED** (§6.2)
4. CartPole as a testbed for C1 — structurally null, risk-neutral and risk-seeking optima coincide. **REFUTED**

9.3 Open

1. **C2 is closed by decision rather than by experiment.** The sample-efficiency argument is structural ($2N_t$ rollouts per update against one backprop) and the paper’s own published

numbers stand for the SPSA side. Recorded here as a deliberate scope choice, not an oversight: a reviewer who wants the head-to-head can have it cheaply on the gridworld, where episodes are three steps. **BY DECISION**

2. **An environment where EVaR beats the alternatives.** This is the binding constraint on the paper. The measures are separable in principle (§6.3), but on every environment tested the arms tie, for reasons now understood in each case. **OPEN**
3. **Adaptive β as the stated contribution.** The scale-adaptive interval is a first instance with a measured failure behind it; the general mechanism is not yet built or evaluated. **OPEN**
4. **λ calibration on Safety-Gymnasium.** Now on the critical path rather than optional: pricing cost is what creates the bimodal return distribution the risk measure needs. Newly feasible because there is finally a trained policy to calibrate against rather than a random walk. **OPEN**
5. **Multi-seed statistics.** Current continuous-control results are 1–2 seeds. Publication needs ≈ 10 seeds with IQM and stratified bootstrap CIs (reliable). **OPEN**
6. **SOTA baselines re-implemented.** IQN with distortion measures, CVaR-AC, DSAC proper — for a like-for-like comparison table. **OPEN**

10 Next steps, in order

1. **Price the cost ($\lambda > 0$) and re-screen.** The screening probe makes this a ~ 30 -minute question per λ : does $\text{sd}(Z)$ rise to a usable fraction of the return scale? Only if it does is an α sweep meaningful.
2. **Stochastic environment variants** with the induced return spread *reported*, since a reviewer will check exactly that.
3. **Adaptive β** , generalising the scale-adaptive interval into a mechanism with an ablation against fixed- β entropic utility.
4. **Scale to the Safe Velocity suite** (Hopper, HalfCheetah, Swimmer, Walker2d, Ant) with 10 seeds once an environment that discriminates is identified.

Appendix A: reproduction

```
# Learner validation (Pendulum, both arms)
python experiments/run_dsac.py --env Pendulum-v1 --risk mean --total-steps 60000 \
    --start-steps 2000 --device cuda --seed 0
python experiments/run_dsac.py --env Pendulum-v1 --risk eva --alpha 0.1 \
    --total-steps 60000 --start-steps 2000 --device cuda --seed 0

# Safety-Gymnasium (separate interpreter; MUJOCO_GL must stay unset)
~/envs/safety/bin/python experiments/run_dsac.py --env SafetyPointGoal1-v0 \
    --risk eva --alpha 0.1 --total-steps 300000 --start-steps 5000 \
    --cost-penalty 0.0 --device cuda --seed 0

# Exact analyses (no learning; numbers are exact)
python analysis/c3_attribution.py          # state- vs action-value regret
python analysis/path_dependent_test.py    # dependence does not break exactness
python analysis/discriminating_env.py     # search for separable configurations

# Figures and the shareable W&B report
python analysis/report_figures.py --out results/figures/report
```

```
python analysis/gridworld_report.py --out results/figures/gridworld
python analysis/dsac_report.py --groups dsac-pendulum dsac-safety-nav \
    --diag-group dsac-safety-probe
```

Appendix B: code map

Path	Contents
<code>evar_deeprl/risk/evar.py</code>	EVaR solver: bisection, envelope gradients, adaptive interval, diagnostics
<code>evar_deeprl/risk/measures.py</code>	Pluggable risk functionals
<code>evar_deeprl/distributional/iqn.q.py</code>	IQN action-value critic $Z(s, a)$; vectorised risk
<code>evar_deeprl/policies/squashed_gaussian.py</code>	Tanh-squashed actor
<code>evar_deeprl/agents/dsac-evar.py</code>	Risk-sensitive DSAC training loop
<code>evar_deeprl/envs/lottery_gridworld.py</code>	Exact testbed with closed-form trajectory EVaR
<code>experiments/run_dsac.py</code>	Runner
<code>analysis/</code>	Exact analyses, figures, W&B report generation

Live dashboards and the interactive version of the DSAC results are on [Weights & Biases](#) in project `evar-deeprl`, groups `dsac-pendulum`, `dsac-safety-nav`, `dsac-safety-probe`.